```
# Prediction Test based on Regression and Tree based method


# import pandas as pd
# import glob

# # Use glob to find all CSV files matching the pattern
# csv_files = glob.glob("mnt-oficial-IJ-*.csv")

# # Create a list to hold the dataframes
# dfs = []

# # Loop through the files and read them into dataframes
# for file in csv_files:
#     df = pd.read_csv(file)
#     dfs.append(df)

# # Concatenate the dataframes
# merged_df = pd.concat(dfs, ignore_index=True)


import pandas as pd

# Load the datasets
df_044 = pd.read_csv("mnt-oficial-IJ-044.csv")
df_046 = pd.read_csv("mnt-oficial-IJ-046.csv")
df_117 = pd.read_csv("mnt-oficial-IJ-117.csv")
df_118 = pd.read_csv("mnt-oficial-IJ-118.csv")
df_119 = pd.read_csv("mnt-oficial-IJ-119.csv")
df_120 = pd.read_csv("mnt-oficial-IJ-120.csv")
df_121 = pd.read_csv("mnt-oficial-IJ-121.csv")
df_122 = pd.read_csv("mnt-oficial-IJ-122.csv")
df_123 = pd.read_csv("mnt-oficial-IJ-123.csv")
df_124 = pd.read_csv("mnt-oficial-IJ-124.csv")
df_125 = pd.read_csv("mnt-oficial-IJ-125.csv")
df_129 = pd.read_csv("mnt-oficial-IJ-129.csv")
df_130 = pd.read_csv("mnt-oficial-IJ-130.csv")
df_131 = pd.read_csv("mnt-oficial-IJ-131.csv")
df_132 = pd.read_csv("mnt-oficial-IJ-132.csv")
df_133 = pd.read_csv("mnt-oficial-IJ-133.csv")
df_134 = pd.read_csv("mnt-oficial-IJ-134.csv")
df_135 = pd.read_csv("mnt-oficial-IJ-135.csv")
df_136 = pd.read_csv("mnt-oficial-IJ-136.csv")
df_137 = pd.read_csv("mnt-oficial-IJ-137.csv")
df_138 = pd.read_csv("mnt-oficial-IJ-138.csv")
df_139 = pd.read_csv("mnt-oficial-IJ-139.csv")
df_151 = pd.read_csv("mnt-oficial-IJ-151.csv")
df_152 = pd.read_csv("mnt-oficial-IJ-152.csv")
df_155 = pd.read_csv("mnt-oficial-IJ-155.csv")
df_156 = pd.read_csv("mnt-oficial-IJ-156.csv")
df_164 = pd.read_csv("mnt-oficial-IJ-164.csv")

# Concatenate the datasets vertically
merged_df = pd.concat(
    [
        df_044,
        df_046,
        df_117,
        df_118,
        df_119,
        df_120,
        df_121,
        df_122,
        df_123,
        df_124,
        df_125,
        df_129,
        df_130,
        df_131,
        df_132,
        df_133,
        df_134,
        df_135,
        df_136,
        df_137,
        df_138,
        df_139,
        df_151,
        df_152,
        df_155,
        df_156,
        df_164,
```

```
    ],
    ignore_index=True,
)


# Display the first 5 rows
print(merged_df.head().to_markdown(index=False, numalign="left", stralign="left"))

# Print the column names and their data types
print(merged_df.info())
```

```
| Cód. Ordem   | Cód. Recurso   | Cód. Produto   | Qtd. Produzida   | Qtd. Refugada   | Qtd. Retrabalhada   | Fator Un.
|:-------------|:---------------|:---------------|:-----------------|:----------------|:--------------------|:----------
| 2680830      | IJ-044         | SA08395        | 868              | 21              | 0                   | 3
| 2680830      | IJ-044         | SA08395        | 1014             | 15              | 0                   | 4
| 2680534      | IJ-044         | SA07321        | 802              | 46              | 0                   | 3
| 2680534      | IJ-044         | SA07321        | 907              | 18              | 0                   | 3
| 2680830      | IJ-044         | SA08395        | 1033             | 16              | 0                   | 5
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8261 entries, 0 to 8260
Data columns (total 16 columns):
 #   Column                                                  Non-Null Count  Dtype
---  ------                                                  --------------  -----
 0   Cód. Ordem                                              8261 non-null   int64
 1   Cód. Recurso                                            8261 non-null   object
 2   Cód. Produto                                            8261 non-null   object
 3   Qtd. Produzida                                          8261 non-null   float64
 4   Qtd. Refugada                                           8261 non-null   float64
 5   Qtd. Retrabalhada                                       8261 non-null   int64
 6   Fator Un.                                               8261 non-null   int64
 7   Cód. Un.                                                8261 non-null   object
 8   Descrição da massa (Composto)                           8261 non-null   object
 9   Consumo de massa no item em (Kg/100pçs)                 8261 non-null   float64
 10  Qtd. Produzida Acumulada Total                          8261 non-null   float64
 11  Qtd. Refugada Acumulada Total                           8261 non-null   float64
 12  Qtd. Retrabalhada Acumulada Total                       8261 non-null   int64
 13  Fator Un. Acumulado Total                               8261 non-null   int64
 14  Consumo de massa no item em (Kg/100pçs) Acumulado Total 8261 non-null   float64
 15  Manutencao                                              8261 non-null   int64
dtypes: float64(6), int64(6), object(4)
memory usage: 1.0+ MB
None
```

```
# Get all unique values from `Cód. Recurso`
unique_values = merged_df["Cód. Recurso"].unique()

# Check the number of unique values in `Cód. Recurso`
if len(unique_values) > 50:
    # If there are too many unique values, sample the top 50
    top_occuring_values = (
        merged_df["Cód. Recurso"].value_counts().head(50).index.tolist()
    )
    print(top_occuring_values)
else:
    # Otherwise print all unique valus in `Cód. Recurso`
    print(unique_values)
```

```
['IJ-044' 'IJ-046' 'IJ-117' 'IJ-118' 'IJ-119' 'IJ-120' 'IJ-121' 'IJ-122'
 'IJ-123' 'IJ-124' 'IJ-125' 'IJ-129' 'IJ-130' 'IJ-131' 'IJ-132' 'IJ-133'
 'IJ-134' 'IJ-135' 'IJ-136' 'IJ-137' 'IJ-138' 'IJ-139' 'IJ-151' 'IJ-152'
 'IJ-155' 'IJ-156' 'IJ-164']
```

```
# Get all unique values from `Cód. Produto`
unique_values = merged_df["Cód. Produto"].unique()

# Check the number of unique values in `Cód. Produto`
if len(unique_values) > 50:
    # If there are too many unique values, sample the top 50
    top_occuring_values = (
        merged_df["Cód. Produto"].value_counts().head(50).index.tolist()
    )
    print(top_occuring_values)
else:
    # Otherwise print all unique valus in `Cód. Produto`
    print(unique_values)

# Get all unique values from `Descrição da massa (Composto)`
unique_values = merged_df["Descrição da massa (Composto)"].unique()

# Check the number of unique values in `Descrição da massa (Composto)`
if len(unique_values) > 50:
    # If there are too many unique values, sample the top 50
    top_occuring_values = (
```

```
        merged_df["Descrição da massa (Composto)"]
        .value_counts()
        .head(50)
        .index.tolist()
    )
    print(top_occuring_values)
else:
    # Otherwise print all unique valus in `Descrição da massa (Composto)`
    print(unique_values)
```

```
[15911, 15909, '12989', '15911', 'SA05003', 15910, '12689', 'SA05266', 'SA04416', '15179', 'SA07050', 'SA02794', '15909'
 ['N-142/67' 'N148/18' 'N146/9' 'N-158/3' 'P212/1' 'F420/306' 'F403/32'
  'F420/96' 'F420/191' 'F424/85' 'P248/40' 'F420/95' 'F420/134' 'F424/40 '
  'F424/40' 'S315/10' 'F420/293' 'S316/2' 'P250/14' 'F444/1' ' F403/32'
  'N144/83' ' F420/191' 'F424/51' 'F424/50' 'F420/201' 'N-142/44' 'N108/17'
  'FP75614/56' 'FP75614/49' 'F424/64' 'TEFLON PTFE 2864 A' 'S-312/1'
  'N-110/2' '403/32' 'N144/85' 'N188/11' ' F420/134' 'AC- 70810' 'N108/72'
  'S315/1' 'RC-26 / 18' ' S315/10' ' F420/95']
```

```
# Make a copy of the DataFrame
df_encoded = merged_df.copy()

# Apply one-hot encoding to the non-numeric columns
df_encoded = pd.get_dummies(
    df_encoded,
    columns=["Cód. Produto", "Descrição da massa (Composto)", "Cód. Recurso"],
)

# Display the first 5 rows of the transformed DataFrame
print(df_encoded.head().to_markdown(index=False, numalign="left", stralign="left"))

# Print the column names and their data types of the transformed DataFrame
print(df_encoded.info())
```

```
| Cód. Ordem   | Qtd. Produzida   | Qtd. Refugada   | Qtd. Retrabalhada   | Fator Un.   | Cód. Un.   | Consumo de massa
|:-------------|:-----------------|:----------------|:--------------------|:------------|:-----------|:-----------------
| 2680830      | 868              | 21              | 0                   | 3           | PC         | 8.607
| 2680830      | 1014             | 15              | 0                   | 4           | PC         | 9.157
| 2680534      | 802              | 46              | 0                   | 3           | PC         | 4.35
| 2680534      | 907              | 18              | 0                   | 3           | PC         | 5.7
| 2680830      | 1033             | 16              | 0                   | 5           | PC         | 12.407
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8261 entries, 0 to 8260
Columns: 522 entries, Cód. Ordem to Cód. Recurso_IJ-164
dtypes: bool(509), float64(6), int64(6), object(1)
memory usage: 4.8+ MB
None
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Drop the `Manutencao` column for the target variable
y = df_encoded["Manutencao"]

# Drop the remaining non-numeric columns and the target variable
X = df_encoded.drop(columns=["Manutencao", "Cód. Un."])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=True  # Add shuffle=True
)


# Create a Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Print the model's score
print("Model score:", model.score(X_test, y_test))
```

```
Model score: 0.9875644841743482
```

```
# Convert `Cód. Produto` and `Descrição da massa (Composto)` to string type
merged_df["Cód. Produto"] = merged_df["Cód. Produto"].astype(str)
merged_df["Descrição da massa (Composto)"] = merged_df[
    "Descrição da massa (Composto)"
].astype(str)
```

```
# Print the column names and their data types
print(merged_df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8261 entries, 0 to 8260
Data columns (total 16 columns):
 #   Column                                                Non-Null Count  Dtype
---  ------                                                --------------  -----
 0   Cód. Ordem                                            8261 non-null   int64
 1   Cód. Recurso                                          8261 non-null   object
 2   Cód. Produto                                          8261 non-null   object
 3   Qtd. Produzida                                        8261 non-null   float64
 4   Qtd. Refugada                                         8261 non-null   float64
 5   Qtd. Retrabalhada                                     8261 non-null   int64
 6   Fator Un.                                             8261 non-null   int64
 7   Cód. Un.                                              8261 non-null   object
 8   Descrição da massa (Composto)                         8261 non-null   object
 9   Consumo de massa no item em (Kg/100pçs)               8261 non-null   float64
 10  Qtd. Produzida Acumulada Total                        8261 non-null   float64
 11  Qtd. Refugada Acumulada Total                         8261 non-null   float64
 12  Qtd. Retrabalhada Acumulada Total                     8261 non-null   int64
 13  Fator Un. Acumulado Total                             8261 non-null   int64
 14  Consumo de massa no item em (Kg/100pçs) Acumulado Total  8261 non-null   float64
 15  Manutencao                                            8261 non-null   int64
dtypes: float64(6), int64(6), object(4)
memory usage: 1.0+ MB
None
```

```
print(merged_df[["Cód. Produto", "Descrição da massa (Composto)"]].dtypes)
```

```
Cód. Produto                     object
Descrição da massa (Composto)    object
dtype: object
```

```
# Apply one-hot encoding to the non-numeric columns
df = pd.get_dummies(
    merged_df,
    columns=[
        "Cód. Produto",
        "Descrição da massa (Composto)",
        "Cód. Recurso",
    ],
)

# Display the first 5 rows of the transformed DataFrame
print(df.head().to_markdown(index=False, numalign="left", stralign="left"))

# Print the column names and their data types of the transformed DataFrame
print(df.info())
```

```
| Cód. Ordem   | Qtd. Produzida   | Qtd. Refugada   | Qtd. Retrabalhada   | Fator Un.   | Cód. Un.   | Consumo de massa
|:-------------|:-----------------|:----------------|:--------------------|:------------|:-----------|:-----------------
| 2680830      | 868              | 21              | 0                   | 3           | PC         | 8.607
| 2680830      | 1014             | 15              | 0                   | 4           | PC         | 9.157
| 2680534      | 802              | 46              | 0                   | 3           | PC         | 4.35
| 2680534      | 907              | 18              | 0                   | 3           | PC         | 5.7
| 2680830      | 1033             | 16              | 0                   | 5           | PC         | 12.407
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8261 entries, 0 to 8260
Columns: 519 entries, Cód. Ordem to Cód. Recurso_IJ-164
dtypes: bool(506), float64(6), int64(6), object(1)
memory usage: 4.8+ MB
None
```

```
# Drop the `Manutencao` column for the target variable
y = df["Manutencao"]

# Drop the remaining non-numeric columns and the target variable
X = df.drop(columns=["Manutencao", "Cód. Un."])


# # Split the data into training and testing sets
# X_train, X_test, y_train, y_test = train_test_split(
#     X, y, test_size=0.2, random_state=42
# )


import numpy as np

X_train = X_train.to_numpy(dtype=np.float32)  # Convert to float32
y_train = y_train.to_numpy(dtype=np.int64)  # Convert to int64
```

```
X_test = X_test.to_numpy(dtype=np.float32)
y_test = y_test.to_numpy(dtype=np.int64)


# Create a Linear Regression model
model = LinearRegression()

# Fit the model to the training data
model.fit(X_train, y_train)

# Print the model's score
print("Model score:", model.score(X_test, y_test))
```

```
Model score: 0.9800494313240051
```

```
print("Model score:", model.score(X_train, y_train))
```

```
Model score: 0.9722833633422852
```

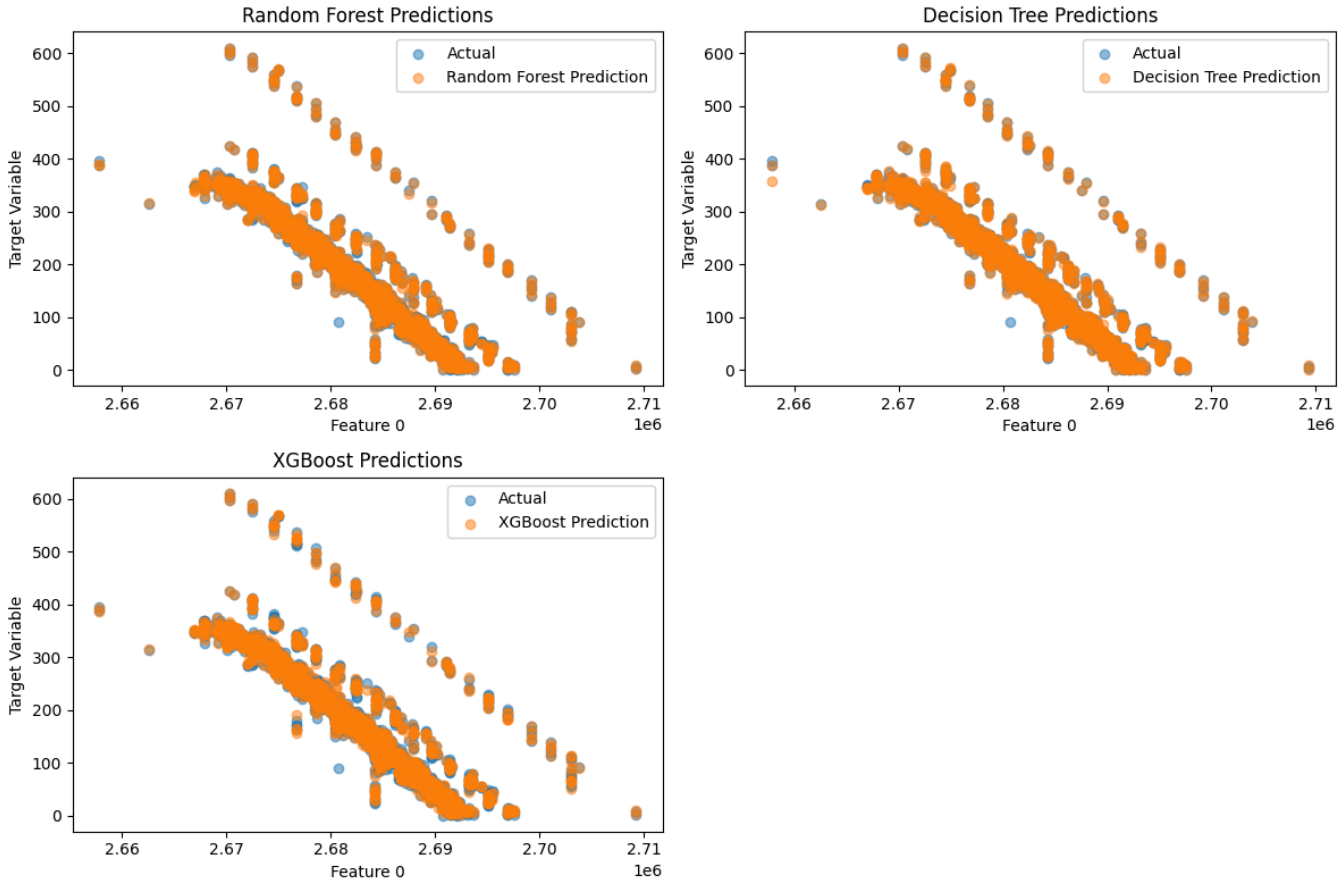
```
for i in range(10):
    random_number = np.random.randint(0, len(X_train))
    prediction = model.predict(X_train[random_number].reshape(1, -1))
    print(f"""Modelo: {prediction}, Real: {y_train[random_number].reshape(-1)}""")
```

```
Modelo: [111.11133], Real: [119]
Modelo: [328.6543], Real: [340]
Modelo: [257.27344], Real: [259]
Modelo: [357.32812], Real: [338]
Modelo: [103.2168], Real: [131]
Modelo: [327.8457], Real: [337]
Modelo: [69.21875], Real: [65]
Modelo: [240.14844], Real: [256]
Modelo: [204.71094], Real: [206]
Modelo: [163.79297], Real: [177]
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
import xgboost as xgb
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.metrics import (
    mean_squared_error,
    r2_score,
    mean_absolute_error,
)  # Import mean_absolute_error


models = {
    "Random Forest": RandomForestRegressor(random_state=42),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "XGBoost": xgb.XGBRegressor(
        random_state=42, objective="reg:squarederror"
    ),  # Important for regression
}

results = {}

for name, model in models.items():
    print(f"Training and evaluating {name}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    # Evaluate the model
    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)

    results[name] = {
        "model": model,
        "mse": mse,
        "r2": r2,
        "mae": mae,  # Added MAE
        "y_pred": y_pred,
        "y_true": y_test,  # Added y_true - CRUCIAL for other metrics!
    }


# Compare the models
print("\nModel Comparison:")
```

```
for name, metrics in results.items():
    print(f"{name}:")
    print(f"  MSE: {metrics['mse']:.4f}")
    print(f"  R-squared: {metrics['r2']:.4f}")
    print(f"  MAE: {metrics['mae']:.4f}")  # Print MAE

# Plotting:
plt.figure(figsize=(12, 8))
for i, (name, metrics) in enumerate(results.items()):
    plt.subplot(2, 2, i + 1)
    plt.scatter(X_test[:, 0], y_test, label="Actual", alpha=0.5)  # Use array indexing
    plt.scatter(
        X_test[:, 0], metrics["y_pred"], label=name + " Prediction", alpha=0.5
    )  # Use array indexing
    plt.xlabel("Feature 0")  # Or a more descriptive name if you know it
    plt.ylabel("Target Variable")
    plt.title(name + " Predictions")
    plt.legend()
plt.tight_layout()
plt.show()
```

```
Training and evaluating Random Forest...
Training and evaluating Decision Tree...
Training and evaluating XGBoost...

Model Comparison:
Random Forest:
  MSE: 12.6871
  R-squared: 0.9991
  MAE: 1.9459
Decision Tree:
  MSE: 20.6213
  R-squared: 0.9985
  MAE: 2.4108
XGBoost:
  MSE: 28.0069
  R-squared: 0.9979
  MAE: 3.6833
```




```
# Example: Accessing the best model (based on R-squared)
best_model_name = max(results, key=lambda k: results[k]["r2"])
best_model = results[best_model_name]["model"]
```

```
print(f"\nBest Model: {best_model_name}")
```

```
Best Model: Random Forest
```

```
# Find the best model based on a chosen metric (e.g., R-squared)
best_model_name = max(results, key=lambda k: results[k]["r2"])
best_model_metrics = results[best_model_name]
# Print the best model's metrics
print(f"\nBest Model: {best_model_name}")
print("Metrics for the best model:")
for metric_name, metric_value in best_model_metrics.items():
    if metric_name != "model":  # Exclude the model object itself
        print(f"  {metric_name}: {metric_value}")
```

```
Best Model: Random Forest
Metrics for the best model:
  mse: 12.68705856019359
  r2: 0.9990540514743952
  mae: 1.9458923169993947
  y_pred: [ 57.89 513.34 167.95 ...  85.3  304.91 165.83]
  y_true: [ 56 513 169 ...  83 308 166]
```

```
results
```

```
{'Random Forest': {'model': RandomForestRegressor(random_state=42),
  'mse': 12.68705856019359,
  'r2': 0.9990540514743952,
  'mae': 1.9458923169993947,
  'y_pred': array([ 57.89, 513.34, 167.95, ...,  85.3 , 304.91, 165.83]),
  'y_true': array([ 56, 513, 169, ...,  83, 308, 166])},
 'Decision Tree': {'model': DecisionTreeRegressor(random_state=42),
  'mse': 20.62129461584997,
  'r2': 0.9984624739339402,
  'mae': 2.410768300060496,
  'y_pred': array([ 87., 514., 168., ...,  84., 307., 167.]),
  'y_true': array([ 56, 513, 169, ...,  83, 308, 166])},
 'XGBoost': {'model': XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
              num_parallel_tree=None, random_state=42, ...),
  'mse': 28.006886676666987,
  'r2': 0.997911810874939,
  'mae': 3.6833154806857102,
  'y_pred': array([ 61.47444, 526.0415 , 168.91058, ...,  85.88519, 306.8573 ,
         167.1964 ], dtype=float32),
  'y_true': array([ 56, 513, 169, ...,  83, 308, 166])}}
```

```
import numpy as np
from sklearn.metrics import (
    mean_squared_error,
    r2_score,
    mean_absolute_error,
    f1_score,
    recall_score,
    precision_score,
    accuracy_score,
)

# ... (Your model training and evaluation code)

for name, metrics in results.items():
    # --- Regression Metrics (already calculated) ---
    print(f"\nEvaluation metrics for {name} (Regression):")
    print(f"  MSE: {metrics['mse']:.4f}")
    print(f"  R-squared: {metrics['r2']:.4f}")
    print(f"  MAE: {metrics['mae']:.4f}")
```

```
Evaluation metrics for Random Forest (Regression):
  MSE: 12.6871
  R-squared: 0.9991
  MAE: 1.9459

Evaluation metrics for Decision Tree (Regression):
```

```
  MSE: 20.6213
  R-squared: 0.9985
  MAE: 2.4108

Evaluation metrics for XGBoost (Regression):
  MSE: 28.0069
  R-squared: 0.9979
  MAE: 3.6833
```

```
for name, model_data in results.items():
    model = model_data["model"]
    predictions = model.predict(X_train)
    print(f"\nPredictions using {name} on random data:")
    print(predictions)
```

```
Predictions using Random Forest on random data:
[342.04 228.36 254.16 ... 251.69 351.36 341.73]

Predictions using Decision Tree on random data:
[340. 228. 254. ... 251. 352. 341.]

Predictions using XGBoost on random data:
[342.23993 237.58339 258.85764 ... 255.81448 350.75504 341.11023]
```

```
print(y_train)
```

```
[340 228 254 ... 251 352 341]
```

```
for name, model_data in results.items():
    model = model_data["model"]
    predictions = model.predict(X_test[:50])
    print(f"\nPredictions using {name} on random data:")
    print(predictions)
```

```
Predictions using Random Forest on random data:
[ 57.89 513.34 167.95 199.19 303.75 160.05 284.7  240.49 135.57  41.83
  50.42 243.47 246.84 162.41 294.6  207.49 351.59 321.25 166.66 248.05
   2.72 222.53 197.84 234.35 326.44 264.67 114.27  42.85 253.75  63.58
 293.16 170.85 322.2   52.01 339.86 284.05 291.28 222.41 162.08   5.63
 514.91 125.41 262.47  19.27 315.22 317.08 124.93 339.35 285.55 100.66]

Predictions using Decision Tree on random data:
[ 87. 514. 168. 202. 298. 163. 283. 239. 131.  42.  50. 243. 245. 165.
 294. 214. 346. 320. 167. 247.   0. 221. 194. 236. 325. 267. 113.  40.
 255.  59. 283. 167. 323.  52. 339. 283. 291. 223. 164.   6. 511. 126.
 260.  19. 319. 313. 127. 340. 286.  98.]

Predictions using XGBoost on random data:
[ 61.47444   526.0415    168.91058   192.72696   310.8899    157.05885
 287.02094   237.04877   138.7863     36.42027    53.478237  241.89076
 248.98488   159.6725    293.35162   212.52625   347.34363   322.20554
 164.93431   253.91899     7.0609956 220.76692   193.90472   233.16396
 321.9753    269.3635    120.39371    42.273876  247.94188    66.10056
 284.03607   170.53166   320.71698    46.626457  343.50574   287.37357
 290.9413    219.7385    152.77464     6.8358197 523.04895   121.125046
 263.8519     19.416286  318.9255    316.20163   119.41868   340.84805
 285.77866   104.42909  ]
```

```
y_test[:50]
```

```
array([ 56, 513, 169, 197, 314, 160, 285, 241, 136,  43,  49, 244, 244,
       164, 293, 209, 347, 325, 172, 245,   1, 222, 197, 235, 326, 266,
       112,  41, 256,  61, 284, 169, 324,  53, 340, 284, 292, 222, 163,
         7, 515, 125, 264,  18, 318, 314, 124, 341, 284,  99])
```

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform  # For integer hyperparameters

models = {
    "Random Forest": RandomForestRegressor(random_state=42),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "XGBoost": xgb.XGBRegressor(random_state=42, objective="reg:squarederror"),
}


param_distributions = {  # Distributions for RandomizedSearchCV
    "Random Forest": {
        "n_estimators": randint(50, 1000),  # Random integers between 50 and 200
        "max_depth": [None, 10, 20, 30, 50, None],  # Include None
        "min_samples_split": randint(2, 8),
        "min_samples_leaf": randint(1, 10),
```

```
        "max_features": ["sqrt", "log2"],
        "bootstrap": [True, False],
    },
    "Decision Tree": {
        "max_depth": [None, 10, 20, 30, 50, None],
        "min_samples_split": randint(2, 8),
        "min_samples_leaf": randint(1, 10),
        "max_features": ["sqrt", "log2"],
        "splitter": ["best", "random"],
    },
    "XGBoost": {
        "n_estimators": randint(50, 500),
        "max_depth": randint(3, 20),
        "learning_rate": [0.001, 0.01, 0.05, 0.1, 0.3, 0.5],
    },
}

# Store the *search objects* (not just the best estimators):
best_models = {}
model_searches = {}  # A dictionary to store the RandomizedSearchCV objects
scoring = ["neg_mean_squared_error", "r2", "neg_mean_absolute_error"]

for name, _ in models.items():  # Iterate over the keys of 'models'
    print(f"Tuning hyperparameters for {name}...")

    # Update the model within the loop
    if name == "Random Forest":
        model = RandomForestRegressor(random_state=42)
    elif name == "Decision Tree":
        model = DecisionTreeRegressor(random_state=42)
    elif name == "XGBoost":
        model = xgb.XGBRegressor(random_state=42, objective="reg:squarederror")
    print(f"Tuning hyperparameters for {name}...")
    random_search = RandomizedSearchCV(
        model,
        param_distributions[name],
        n_iter=20,  # Number of random combinations to try
        cv=10,
        scoring=scoring,  # Use appropriate scoring
        refit="neg_mean_squared_error",  # Specify which metric to use for refitting the best model
        n_jobs=-1,
        random_state=42,  # For reproducibility
    )
    random_search.fit(X_train, y_train)
    model_searches[name] = random_search  # Store the *search object*
    best_models[name] = random_search.best_estimator_

    print(f"Best hyperparameters for {name}: {random_search.best_params_}")
    print(f"Best score for {name}: {random_search.best_score_}")
```

```
Tuning hyperparameters for Random Forest...
Tuning hyperparameters for Random Forest...
Best hyperparameters for Random Forest: {'bootstrap': False, 'max_depth': None, 'max_features': 'sqrt', 'min_samples_lea
Best score for Random Forest: -159.4720775498492
Tuning hyperparameters for Decision Tree...
Tuning hyperparameters for Decision Tree...
Best hyperparameters for Decision Tree: {'max_depth': 20, 'max_features': 'sqrt', 'min_samples_leaf': 3, 'min_samples_sp
Best score for Decision Tree: -2014.915317710631
Tuning hyperparameters for XGBoost...
Tuning hyperparameters for XGBoost...
Best hyperparameters for XGBoost: {'learning_rate': 0.1, 'max_depth': 10, 'n_estimators': 480}
Best score for XGBoost: -13.124895530578602
```

```
print(f"Best hyperparameters for {name}: {random_search.best_params_}")
print(f"Best score for {name}: {random_search.best_score_}")
```

```
Best hyperparameters for XGBoost: {'learning_rate': 0.1, 'max_depth': 10, 'n_estimators': 480}
Best score for XGBoost: -13.124895530578602
```

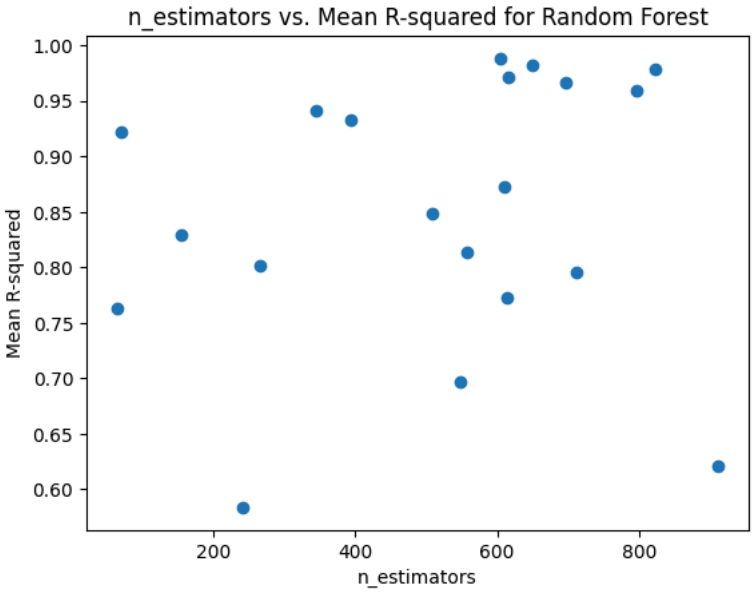
```
import seaborn as sns
import plotly.express as px

for name, model_search in model_searches.items():
    print(f"Results for {name}:")
    results_df = pd.DataFrame(model_search.cv_results_)
    print(results_df.head())

    # Find the best row (highest score)
    best_row = results_df.loc[results_df["mean_test_r2"].idxmax()]
    print(f"Best row:\n{best_row}")
```

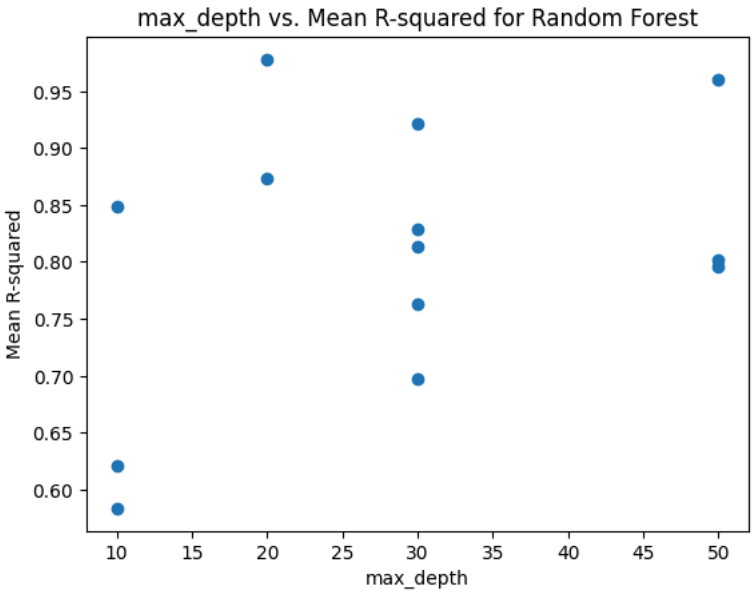
```
    # Scatter plot (n_estimators vs. R-squared)
    if "param_n_estimators" in results_df.columns:
        plt.figure()
        plt.scatter(results_df["param_n_estimators"], results_df["mean_test_r2"])
        plt.xlabel("n_estimators")
        plt.ylabel("Mean R-squared")
        plt.title(f"n_estimators vs. Mean R-squared for {name}")
        plt.show()

        # max_depth vs. R-squared
    if "param_max_depth" in results_df.columns:
        plt.figure()
        plt.scatter(results_df["param_max_depth"], results_df["mean_test_r2"])
        plt.xlabel("max_depth")
        plt.ylabel("Mean R-squared")
        plt.title(f"max_depth vs. Mean R-squared for {name}")
        plt.show()

    # min_samples_split vs. R-squared (if applicable)
    if "param_min_samples_split" in results_df.columns:
        plt.figure()
        plt.scatter(results_df["param_min_samples_split"], results_df["mean_test_r2"])
        plt.xlabel("min_samples_split")
        plt.ylabel("Mean R-squared")
        plt.title(f"min_samples_split vs. Mean R-squared for {name}")
        plt.show()

    # learning_rate vs. R-squared (if applicable)
    if "param_learning_rate" in results_df.columns:
        plt.figure()
        plt.scatter(results_df["param_learning_rate"], results_df["mean_test_r2"])
        plt.xlabel("learning_rate")
        plt.ylabel("Mean R-squared")
        plt.title(f"learning_rate vs. Mean R-squared for {name}")
        plt.xscale("log")  # Use logarithmic scale for learning rate
        plt.show()

    # Parallel Coordinates Plot (using R-squared for color)
    fig = px.parallel_coordinates(
        results_df,
        color="mean_test_r2",
        title=f"Parallel Coordinates Plot for {name}",
    )
    fig.show()

    # Heatmap (using R-squared for values)
    if (
        "param_n_estimators" in results_df.columns
        and "param_max_depth" in results_df.columns
    ):
        pivoted_results = results_df.pivot_table(
            values="mean_test_r2",
            index="param_n_estimators",
            columns="param_max_depth",
        )
        plt.figure()
        sns.heatmap(pivoted_results, annot=True, cmap="viridis")
        plt.title(f"Heatmap of n_estimators vs. max_depth for {name}")
        plt.xlabel("max_depth")
        plt.ylabel("n_estimators")
        plt.show()

    if name == "Random Forest":
        # Heatmap for n_estimators vs. max_depth (Random Forest)
        if (
            "param_n_estimators" in results_df.columns
            and "param_max_depth" in results_df.columns
        ):
            pivoted_results = results_df.pivot_table(
                values="mean_test_r2",
                index="param_n_estimators",
                columns="param_max_depth",
            )
            plt.figure()
            sns.heatmap(pivoted_results, annot=True, cmap="viridis")
            plt.title(f"Heatmap of n_estimators vs. max_depth for {name}")
            plt.xlabel("max_depth")
            plt.ylabel("n_estimators")
            plt.show()

    elif name == "Decision Tree":
        # Heatmap for max_depth vs. min_samples_split (Decision Tree)
```

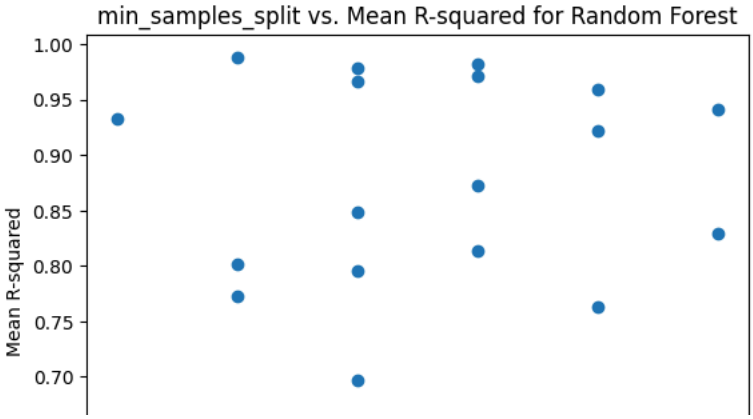
```
        if (
            "param_max_depth" in results_df.columns
            and "param_min_samples_split" in results_df.columns
        ):
            pivoted_results = results_df.pivot_table(
                values="mean_test_r2",
                index="param_max_depth",
                columns="param_min_samples_split",
            )
            plt.figure()
            sns.heatmap(pivoted_results, annot=True, cmap="viridis")
            plt.title(f"Heatmap of max_depth vs. min_samples_split for {name}")
            plt.xlabel("min_samples_split")
            plt.ylabel("max_depth")
            plt.show()

    elif name == "XGBoost":
        # Heatmap for learning_rate vs. max_depth (XGBoost)
        if (
            "param_learning_rate" in results_df.columns
            and "param_max_depth" in results_df.columns
        ):
            pivoted_results = results_df.pivot_table(
                values="mean_test_r2",
                index="param_learning_rate",
                columns="param_max_depth",
            )
            plt.figure()
            sns.heatmap(pivoted_results, annot=True, cmap="viridis")
            plt.title(f"Heatmap of learning_rate vs. max_depth for {name}")
            plt.xlabel("max_depth")
            plt.ylabel("learning_rate")
            plt.show()
```

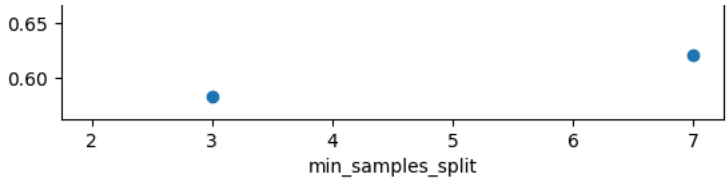
```
Results for Random Forest:
   mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
0       1.720874      0.066731         0.028918        0.005499
1       9.677280      0.230411         0.136698        0.011820
2      18.302944      1.803978         0.335781        0.081235
3       3.707379      0.859621         0.102425        0.028168
4      42.444358      1.456367         0.430072        0.070610

   param_bootstrap param_max_depth param_max_features  param_min_samples_leaf  \
0             True              30               sqrt                       8
1             True              10               sqrt                       7
2            False              50               log2                       8
3             True              10               log2                       6
4            False              50               sqrt                       6

   param_min_samples_split  param_n_estimators  ...  \
0                        6                  70  ...
1                        4                 508  ...
2                        4                 711  ...
3                        3                 241  ...
4                        6                 797  ...

  split3_test_neg_mean_absolute_error  split4_test_neg_mean_absolute_error  \
0                          -23.871481                          -22.910556
1                          -34.746265                          -33.168085
2                          -40.432331                          -39.176305
3                          -60.894251                          -57.297897
4                          -16.587923                          -16.398745

   split5_test_neg_mean_absolute_error  split6_test_neg_mean_absolute_error  \
0                          -21.969821                          -24.318900
1                          -30.742202                          -34.419313
2                          -38.055516                          -40.498402
3                          -55.065145                          -58.206574
4                          -16.093645                          -16.913336

   split7_test_neg_mean_absolute_error  split8_test_neg_mean_absolute_error  \
0                          -24.339650                          -23.943404
1                          -35.707670                          -33.777229
2                          -40.660334                          -42.090739
3                          -60.176740                          -61.468217
4                          -16.993925                          -16.977676

   split9_test_neg_mean_absolute_error  mean_test_neg_mean_absolute_error  \
0                          -24.823818                         -23.736187
1                          -37.863147                         -34.525223
2                          -42.985723                         -40.644034
3                          -63.322839                         -59.581223
4                          -18.315926                         -16.933916

   std_test_neg_mean_absolute_error  rank_test_neg_mean_absolute_error
0                          1.069511                                  9
1                          2.065639                                 11
2                          1.533938                                 15
3                          2.427307                                 20
4                          0.702497                                  6

[5 rows x 50 columns]
Best row:
mean_fit_time                                                          38.902307
std_fit_time                                                            0.888151
mean_score_time                                                         0.521407
std_score_time                                                          0.101028
param_bootstrap                                                            False
param_max_depth                                                             None
param_max_features                                                          sqrt
param_min_samples_leaf                                                         2
param_min_samples_split                                                        3
param_n_estimators                                                           605
params                                 {'bootstrap': False, 'max_depth': None, 'max_f...
split0_test_neg_mean_squared_error                                     -160.471029
split1_test_neg_mean_squared_error                                     -195.530168
split2_test_neg_mean_squared_error                                     -138.231425
split3_test_neg_mean_squared_error                                     -150.930054
split4_test_neg_mean_squared_error                                     -128.084145
split5_test_neg_mean_squared_error                                     -134.098627
split6_test_neg_mean_squared_error                                     -153.678323
split7_test_neg_mean_squared_error                                     -187.255973
split8_test_neg_mean_squared_error                                     -144.555314
split9_test_neg_mean_squared_error                                     -201.885717
mean_test_neg_mean_squared_error                                       -159.472078
std_test_neg_mean_squared_error                                          25.062459
rank_test_neg_mean_squared_error                                                 1
split0_test_r2                                                            0.98876
split1_test_r2                                                           0.986461
split2_test_r2                                                           0.988909
split3_test_r2                                                           0.988634
split4_test_r2                                                           0.990205
split5_test_r2                                                           0.989794
split6_test_r2                                                           0.988671
```

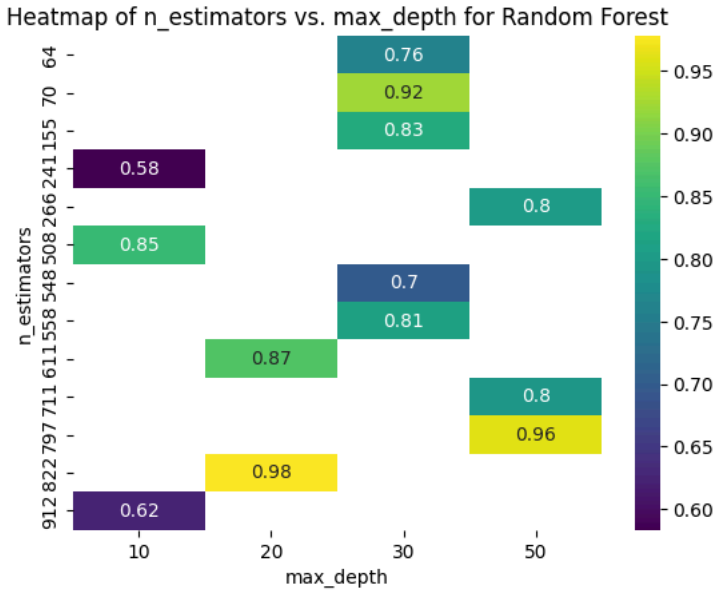
```
split7_test_r2                                                    0.98631
split8_test_r2                                                   0.989332
split9_test_r2                                                   0.986551
mean_test_r2                                                     0.988363
std_test_r2                                                      0.001346
rank_test_r2                                                            1
split0_test_neg_mean_absolute_error                             -9.239655
split1_test_neg_mean_absolute_error                             -9.312117
split2_test_neg_mean_absolute_error                             -8.368267
split3_test_neg_mean_absolute_error                             -8.674263
split4_test_neg_mean_absolute_error                             -8.225102
split5_test_neg_mean_absolute_error                             -8.275043
split6_test_neg_mean_absolute_error                             -8.992347
split7_test_neg_mean_absolute_error                             -9.134041
split8_test_neg_mean_absolute_error                             -8.974728
split9_test_neg_mean_absolute_error                             -9.585079
mean_test_neg_mean_absolute_error                               -8.878064
std_test_neg_mean_absolute_error                                 0.447031
rank_test_neg_mean_absolute_error                                       1
Name: 12, dtype: object
```

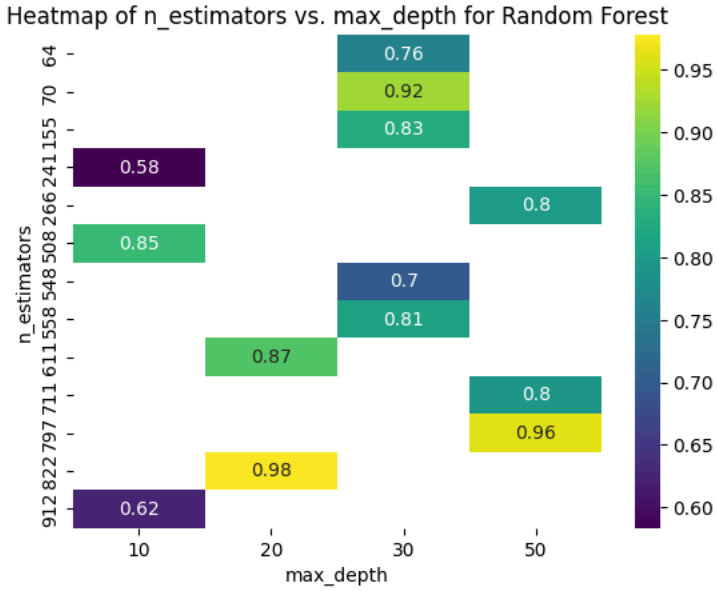


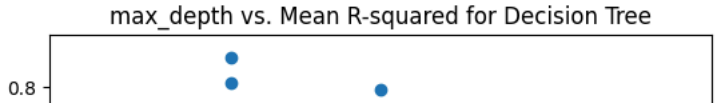


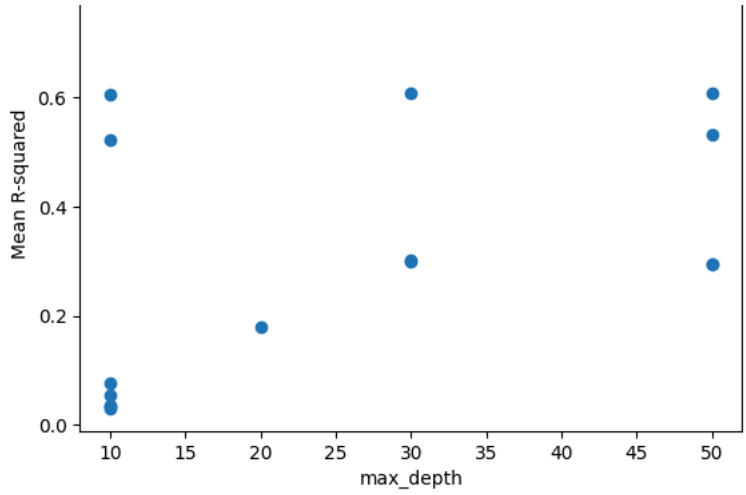



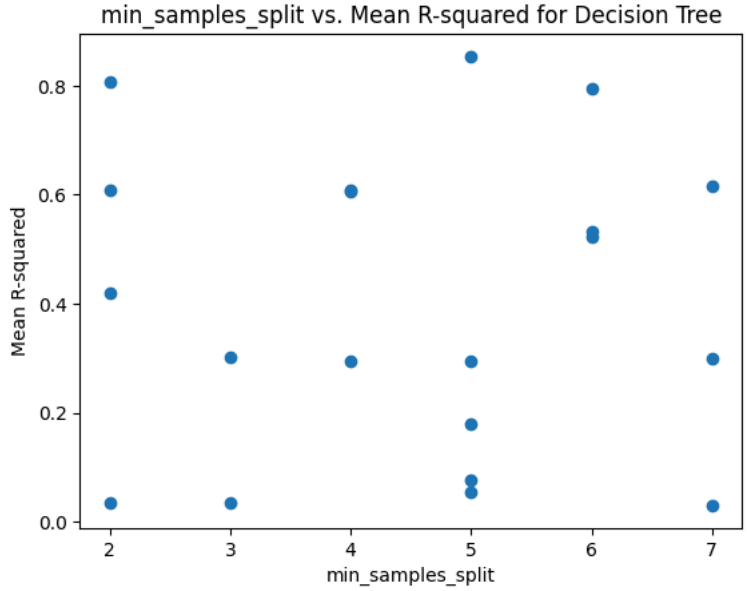



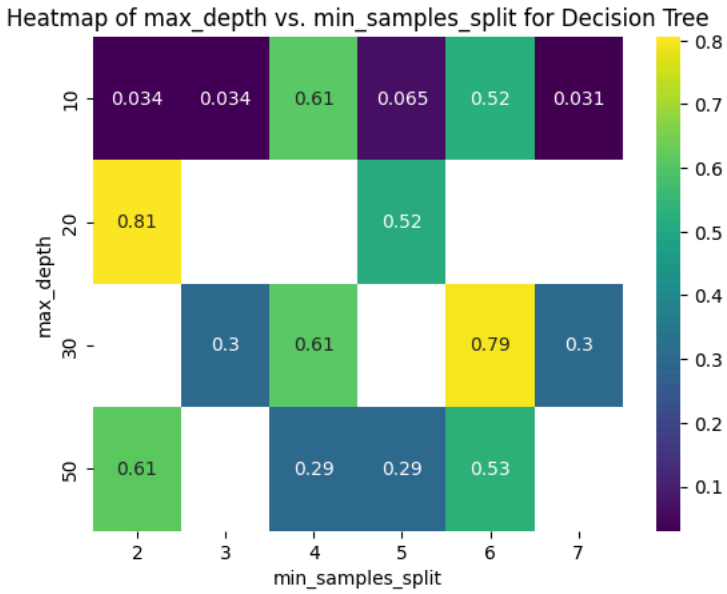

```
Results for Decision Tree:
   mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
0       0.072019      0.009118         0.004505        0.001503
1       0.050166      0.007072         0.005143        0.001772
2       0.060642      0.015647         0.004727        0.001270
3       0.071486      0.023415         0.008234        0.004851
4       0.074134      0.021212         0.005282        0.002193

  param_max_depth param_max_features  param_min_samples_leaf  \
0              30               sqrt                       8
1              10               sqrt                       7
2              50               log2                       8
3              50               log2                       8
4            None               log2                       5

   param_min_samples_split param_splitter  \
0                        6           best
1                        4           best
2                        4         random
3                        5         random
4                        2         random

                                              params  ...  \
0  {'max_depth': 30, 'max_features': 'sqrt', 'min...  ...
1  {'max_depth': 10, 'max_features': 'sqrt', 'min...  ...
2  {'max_depth': 50, 'max_features': 'log2', 'min...  ...
3  {'max_depth': 50, 'max_features': 'log2', 'min...  ...
4  {'max_depth': None, 'max_features': 'log2', 'm...  ...

   split3_test_neg_mean_absolute_error  split4_test_neg_mean_absolute_error  \
```

```
0                          -39.714778                          -33.448843
1                          -60.862217                          -47.949682
2                          -70.800021                          -61.242519
3                          -70.800021                          -61.242519
4                          -72.991116                          -60.546782

   split5_test_neg_mean_absolute_error  split6_test_neg_mean_absolute_error  \
0                          -33.764437                          -42.418790
1                          -59.265723                          -64.888037
2                          -88.763005                          -58.462724
3                          -88.763005                          -58.462724
4                          -59.372564                          -56.524454

   split7_test_neg_mean_absolute_error  split8_test_neg_mean_absolute_error  \
0                          -39.214490                          -32.392697
1                          -73.882346                          -39.833060
2                          -78.769518                          -79.018659
3                          -78.769518                          -79.018659
4                          -62.457395                          -75.265212

   split9_test_neg_mean_absolute_error  mean_test_neg_mean_absolute_error  \
0                          -36.039937                         -35.923073
1                          -61.873099                         -56.027278
2                          -83.485559                         -78.162480
3                          -83.485559                         -78.162480
4                          -92.334923                         -68.608528

   std_test_neg_mean_absolute_error  rank_test_neg_mean_absolute_error
0                          3.285545                                  3
1                         11.472961                                  7
2                         10.981425                                 13
3                         10.981425                                 13
4                         11.557552                                 10

[5 rows x 49 columns]
Best row:
mean_fit_time                                                          0.063788
std_fit_time                                                           0.006798
mean_score_time                                                        0.004382
std_score_time                                                         0.001146
param_max_depth                                                              20
param_max_features                                                         sqrt
param_min_samples_leaf                                                        3
param_min_samples_split                                                       5
param_splitter                                                             best
params                                 {'max_depth': 20, 'max_features': 'sqrt', 'min...
split0_test_neg_mean_squared_error                                    -4191.347469
split1_test_neg_mean_squared_error                                    -1943.418288
split2_test_neg_mean_squared_error                                     -845.288708
split3_test_neg_mean_squared_error                                    -3683.919672
split4_test_neg_mean_squared_error                                      -792.38703
split5_test_neg_mean_squared_error                                    -2563.021472
split6_test_neg_mean_squared_error                                    -1101.618453
split7_test_neg_mean_squared_error                                    -1337.837613
split8_test_neg_mean_squared_error                                    -1292.031041
split9_test_neg_mean_squared_error                                    -2398.283429
mean_test_neg_mean_squared_error                                      -2014.915318
std_test_neg_mean_squared_error                                        1124.839749
rank_test_neg_mean_squared_error                                                 1
split0_test_r2                                                             0.70642
split1_test_r2                                                             0.86543
split2_test_r2                                                            0.932177
split3_test_r2                                                            0.722588
split4_test_r2                                                            0.939405
split5_test_r2                                                            0.804931
split6_test_r2                                                            0.918789
split7_test_r2                                                            0.902193
split8_test_r2                                                            0.904646
split9_test_r2                                                            0.840234
mean_test_r2                                                              0.853681
std_test_r2                                                               0.080166
rank_test_r2                                                                     1
split0_test_neg_mean_absolute_error                                      -44.373631
split1_test_neg_mean_absolute_error                                      -29.853952
split2_test_neg_mean_absolute_error                                      -19.941718
split3_test_neg_mean_absolute_error                                      -40.502126
split4_test_neg_mean_absolute_error                                      -18.745913
split5_test_neg_mean_absolute_error                                      -34.536067
split6_test_neg_mean_absolute_error                                        -25.0265
split7_test_neg_mean_absolute_error                                      -26.003115
split8_test_neg_mean_absolute_error                                      -24.643698
split9_test_neg_mean_absolute_error                                      -33.706105
mean_test_neg_mean_absolute_error                                        -29.733283
std_test_neg_mean_absolute_error                                           8.048237
rank_test_neg_mean_absolute_error                                                1
Name: 6, dtype: object
```





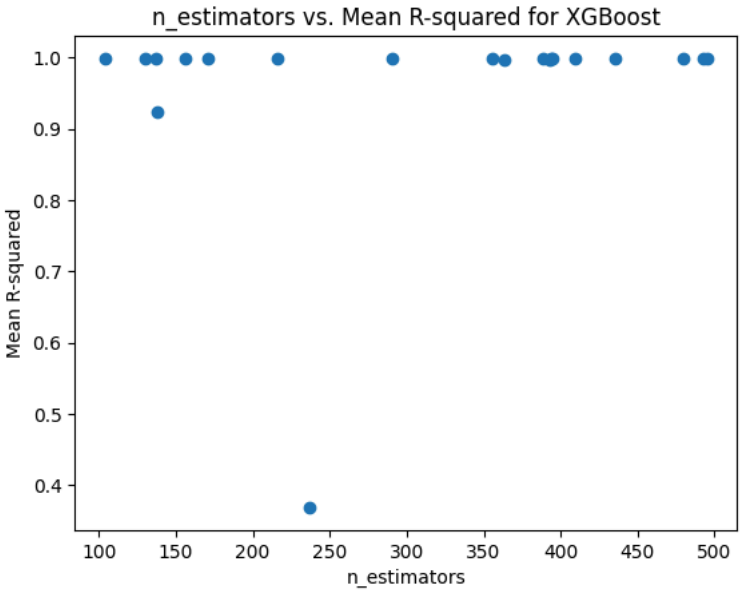



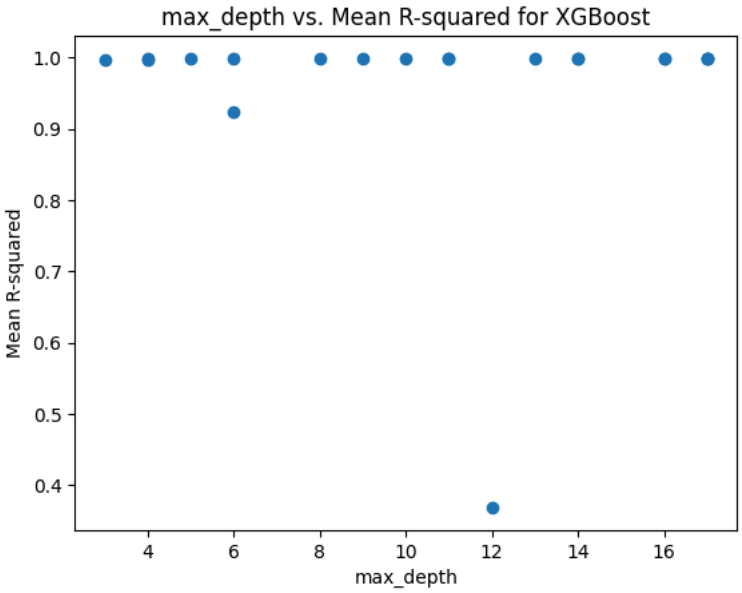



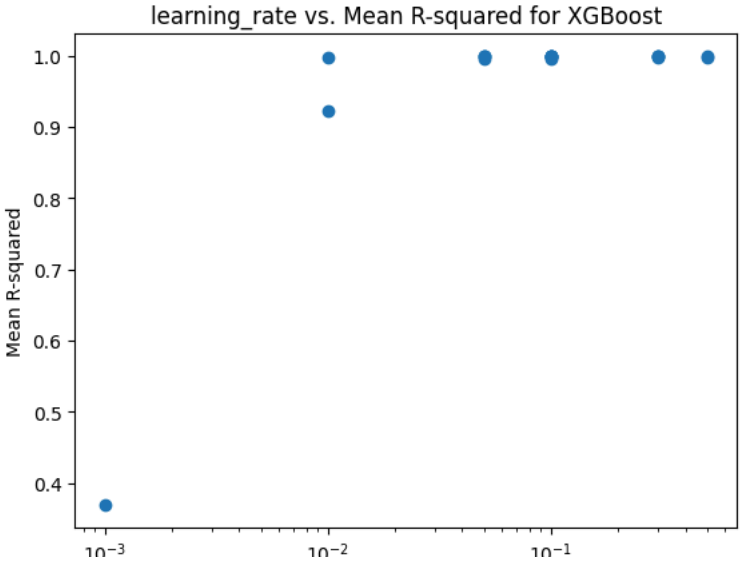

```
Results for XGBoost:
   mean_fit_time  std_fit_time  mean_score_time  std_score_time  \
0      19.867169      2.154097         0.056924        0.007752
1       5.147953      0.261731         0.023288        0.002040
2      19.014986      0.282180         0.038311        0.006451
3       8.798141      0.078842         0.034485        0.004518
4       7.224909      0.153544         0.024190        0.003898

   param_learning_rate  param_max_depth  param_n_estimators  \
0                 0.10               17                 156
1                 0.30                9                 171
2                 0.05               13                 137
3                 0.30                6                 409
4                 0.05                4                 393

                                              params  \
0  {'learning_rate': 0.1, 'max_depth': 17, 'n_est...
```

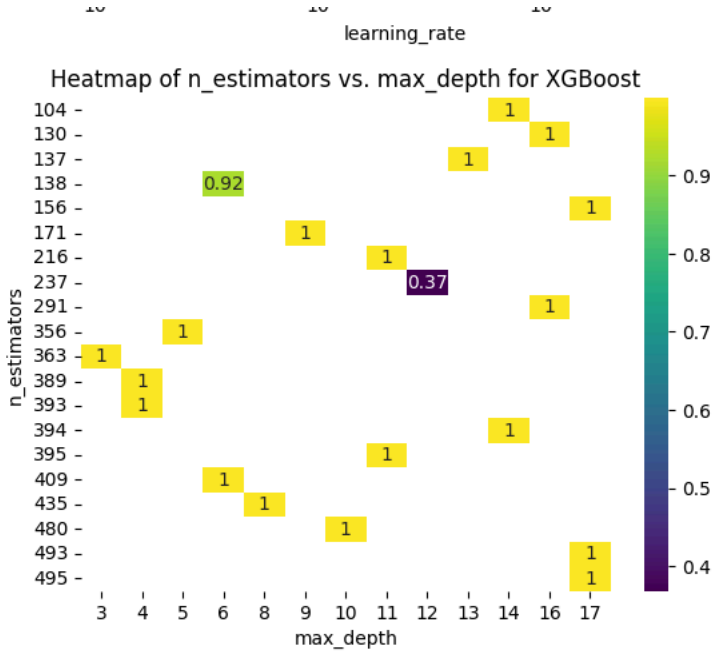
```
1  {'learning_rate': 0.3, 'max_depth': 9, 'n_esti...
2  {'learning_rate': 0.05, 'max_depth': 13, 'n_es...
3  {'learning_rate': 0.3, 'max_depth': 6, 'n_esti...
4  {'learning_rate': 0.05, 'max_depth': 4, 'n_est...

   split0_test_neg_mean_squared_error  split1_test_neg_mean_squared_error  \
0                          -13.508592                           -9.234868
1                          -15.970479                          -13.518474
2                          -13.454584                           -9.713334
3                          -16.808790                          -15.380695
4                          -53.567191                          -48.957681

   ...  split3_test_neg_mean_absolute_error  \
0  ...                            -2.246582
1  ...                            -2.716562
2  ...                            -2.429876
3  ...                            -3.096554
4  ...                            -5.185555

   split4_test_neg_mean_absolute_error  split5_test_neg_mean_absolute_error  \
0                            -2.008906                            -2.149271
1                            -2.535095                            -2.607939
2                            -2.102316                            -2.219299
3                            -2.774673                            -2.935003
4                            -4.924603                            -4.703062

   split6_test_neg_mean_absolute_error  split7_test_neg_mean_absolute_error  \
0                            -2.226169                            -2.191861
1                            -2.700900                            -2.637386
2                            -2.252900                            -2.164785
3                            -2.916549                            -2.737121
4                            -5.272052                            -4.913418

   split8_test_neg_mean_absolute_error  split9_test_neg_mean_absolute_error  \
0                            -2.245125                            -2.424991
1                            -2.699891                            -2.938692
2                            -2.304221                            -2.503768
3                            -2.896093                            -2.905239
4                            -5.066480                            -5.257259

   mean_test_neg_mean_absolute_error  std_test_neg_mean_absolute_error  \
0                          -2.212388                          0.127608
1                          -2.676087                          0.110632
2                          -2.272433                          0.131822
3                          -2.882058                          0.098205
4                          -5.104359                          0.203207

   rank_test_neg_mean_absolute_error
0                                  2
1                                 10
2                                  6
3                                 11
4                                 17

[5 rows x 47 columns]
Best row:
mean_fit_time                                                          16.03352
std_fit_time                                                           0.261147
mean_score_time                                                        0.057359
std_score_time                                                         0.009027
param_learning_rate                                                         0.1
param_max_depth                                                              10
param_n_estimators                                                          480
params                                 {'learning_rate': 0.1, 'max_depth': 10, 'n_est...
split0_test_neg_mean_squared_error                                    -12.724228
split1_test_neg_mean_squared_error                                      -9.28898
split2_test_neg_mean_squared_error                                    -11.909096
split3_test_neg_mean_squared_error                                    -14.173427
split4_test_neg_mean_squared_error                                    -10.173252
split5_test_neg_mean_squared_error                                     -12.78511
split6_test_neg_mean_squared_error                                    -18.170705
split7_test_neg_mean_squared_error                                    -10.830635
split8_test_neg_mean_squared_error                                    -12.219208
split9_test_neg_mean_squared_error                                    -18.974316
mean_test_neg_mean_squared_error                                      -13.124896
std_test_neg_mean_squared_error                                         3.033807
rank_test_neg_mean_squared_error                                               1
split0_test_r2                                                          0.999109
split1_test_r2                                                          0.999357
split2_test_r2                                                          0.999044
split3_test_r2                                                          0.998933
split4_test_r2                                                          0.999222
split5_test_r2                                                          0.999027
split6_test_r2                                                           0.99866
split7_test_r2                                                          0.999208
split8_test_r2                                                          0.999098
split9_test_r2                                                          0.998736
mean_test_r2                                                            0.999039
std_test_r2                                                             0.000204
rank_test_r2                                                                   1
```

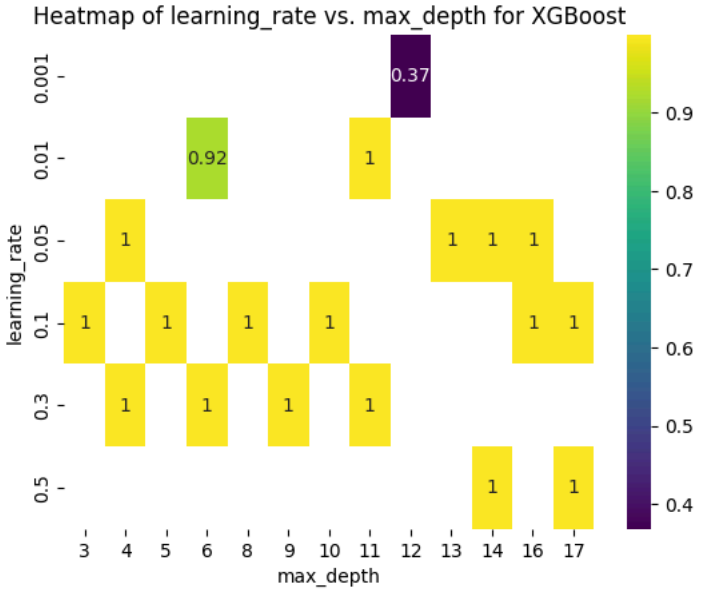
```
split0_test_neg_mean_absolute_error                    -2.400833
split1_test_neg_mean_absolute_error                      -2.2301
split2_test_neg_mean_absolute_error                    -2.186841
split3_test_neg_mean_absolute_error                    -2.327432
split4_test_neg_mean_absolute_error                    -2.118302
split5_test_neg_mean_absolute_error                    -2.236196
split6_test_neg_mean_absolute_error                    -2.222678
split7_test_neg_mean_absolute_error                    -2.212758
split8_test_neg_mean_absolute_error                    -2.276522
split9_test_neg_mean_absolute_error                    -2.465182
mean_test_neg_mean_absolute_error                      -2.267684
std_test_neg_mean_absolute_error                        0.098325
rank_test_neg_mean_absolute_error                              5
Name: 18, dtype: object
```




learning_rate
Heatmap of n_estimators vs. max_depth for XGBoost
n_estimators
104
130
137
138
156
171
216
237
291
356
363
389
393
394
395
409
435
480
493
495
max_depth
3 4 5 6 8 9 10 11 12 13 14 16 17
0.92
0.37
0.9
0.8
0.7
0.6
0.5
0.4


Heatmap of learning_rate vs. max_depth for XGBoost
learning_rate
0.001
0.01
0.05
0.1
0.3
0.5
max_depth
3 4 5 6 8 9 10 11 12 13 14 16 17
0.37
0.92
0.9
0.8
0.7
0.6
0.5
0.4